

Aval

Prove money is committed but not yet arrived, so a counterparty can act now instead of waiting for settlement.

Onatola Timilehin Faruq · github.com/Leihyn/aval

The dead window

\$2M leaves Ethereum at 09:00. It arrives at 09:41. For 41 minutes the money exists, it is irrevocably committed, and it is useless.

The counterparty will not release their side against a screenshot, so both desks wait. This is the largest source of idle working capital in cross-chain finance, and it exists for one reason: there is a gap between a fact being TRUE and that fact being PUBLICLY VERIFIABLE.

Wait 41 minutes, or reveal your position. Those are the only two options today.

Prove the threshold, not the number

Alice locks funds on a source chain. She proves to the counterparty's Midnight contract that the locked amount clears their threshold and is earmarked for them, without revealing the amount, the lock, or her identity.

STAYS PRIVATE (WITNESS)

- lock_id
- amount
- salt
- merkle path

BECOMES PUBLIC (LEDGER)

- merkle root
- nullifier
- fill counter
- attestor id

Six disclose() sites bridge the two. Compact refuses to compile if a witness reaches the ledger without one.

How it works



The attestor never learns anything Alice does not already send it. The chain never learns the amount.

WHY MIDNIGHT SPECIFICALLY

This needs persistent private state on-chain, ZK predicate proofs over it, and a public ledger entry a counterparty contract can gate on. Midnight's dual ledger is that triple. On an EVM you need FHE or an off-chain prover, and you lose the persistent private state.

It compiles, and it is tested

COMPACT COMPILE

```
$ compact compile src/inflight.compact out-full
Compiling 2 circuits:
$ ls out-full/keys/
prove_funds_in_flight.prover
prove_funds_in_flight.verifier
register_attestation.prover
register_attestation.verifier
```

NPM TEST

Tests 25 passed (25)

Duration 1.26s

No Docker. No proof server.
No wallet. Clone and run.

LEDGER STATE AFTER ONE REAL PROOF

```
$ npx tsx scripts/seed-demo.ts
--- Aval demo seed: produced by real circuit execution ---
attestor registered : true
attestations in tree: 3
fills               : 1
nullifiers spent    : 1
nullifier           : 7fe3e3af80ac5555e7a8980793ed8432e0a78857934d5177ad25937f8eaea0f5
amount in ledger    : absent by construction
```

Can an observer tell 50,000 from 5,000,000?

Same lock id. Same salt. Same counterparty. Amounts a hundredfold apart.

REAL OUTPUT, SIDE BY SIDE

```
$ npx tsx scripts/indistinguishability.ts
```

```
--- Aval: can an observer tell 50,000 from 5,000,000? ---
```

public field	amount = 50,000	amount = 5,000,000	same?
attestor	06b9adbc74b16b63..	06b9adbc74b16b63..	IDENTICAL
fills	1	1	IDENTICAL
nullifier	913026c16dbac790..	913026c16dbac790..	IDENTICAL
spent_size	1	1	IDENTICAL
merkle_root	1569299046839186..	1556834036681705..	DIFFERS

Every readable field is identical except the merkle root.

A root is a hash: it commits to the leaf without revealing it. The nullifier is byte-identical across the difference.

Not an absence you have to trust. An equality you can run.

The product, running in a browser

Real screenshot. Circuits execute in the page against real ledger state.

- Midnight runtime ready. Circuits are executing in this browser tab, against real ledger state.

• WHAT THE COUNTERPARTY LEARNS

Release authorized

A threshold of **1,500,000** was cleared.

In this demo Alice holds **2,000,000**. Ask for less and the proof clears; ask for more and it is rejected. Either way the counterparty learns only whether the bar was cleared, never the figure.

REQUIRED THRESHOLD

1,500,000

Prove funds in flight

THEY LEARN

- the threshold was cleared
- a registered attestor vouched for it
- it is single-use and expires

THEY NEVER LEARN

- the amount held
- which lock, or which attestation
- anything a third party could reuse

• WHAT THE PUBLIC CHAIN LEARNS

AMOUNT

the figure the counterparty acts on

not present

ATTESTOR REGISTERED

is any attestor trusted

true

ATTESTOR ID

which institution, not which customer

06b9adbc74b16b6310ac8d69...

MERKLE ROOT

binds the attestation, hides it

491255682427379505727498...

NULLIFIERS

blocks proof reuse

0016a476e02417c12b47e120...

FILLS

how many cleared, never how much

1

PRIVATE CIRCUIT INPUTS, NEVER WRITTEN

- amount
- lock id
- salt
- merkle path
- prover identity

This list is re-checked against the live ledger object on every update. If one of these ever appeared on chain, this pane would say so.

Six attacks, six reverts from the circuit

REAL OUTPUT

```
reuse the same lock twice      BLOCKED: this lock has already backed a proof
inflate the claimed amount     BLOCKED: no merkle path: leaf is not registered
redirect to a different payee  BLOCKED: no merkle path: leaf is not registered
extend your own expiry        BLOCKED: no merkle path: leaf is not registered
prove after expiry            BLOCKED: attestation expired
ask for more than is locked    BLOCKED: locked amount below required threshold
All six blocked by the circuit, not by application code.
```

One lock backs one proof	nullifier from private lock_id + salt	4 tests
Attestation not transferable	counterparty + expiry hashed into the leaf	3 tests
Cannot inflate the amount	leaf recomputed in-circuit	1 test
Attestations expire	kernel.blockTimeLessThan, ledger time	2 tests
Amount never public	asserted against a full ledger dump	4 tests

The honest part

Midnight cannot see Ethereum. There is no trustless answer without a light client, so Aval does not pretend otherwise. It makes the trusted party someone whose trust is free.

WHAT SHIPS IN WAVE 1

The counterparty runs the attesor themselves. Their contract cannot see Ethereum even though they can.
Trust collapses to "Bob trusts Bob's own node", which is not a trust assumption at all.

ROADMAP 1

k-of-n quorum
small multilateral

ROADMAP 2

bonded + slashable
a fabricated lockId is provably contradicted on
the source chain

ROADMAP 3

light client
actual trustlessness

Residual: a malicious attesor can fabricate a lock. Same trust model as every fast-finality bridge shipping today.
Aval adds privacy to that model. It does not claim to beat it.

One predicate today. The primitive is general.

Wave 1 SHIPPED

funds in flight

`amount >= required AND beneficiary == counterparty`

Wave 2

identity preconditions

`kyc_passed AND jurisdiction NOT IN sanctioned`

Wave 3

private solvency

`reserves >= liabilities`

Wave 3

invoice factoring

`invoice_valid AND unpaid AND amount >= advance`

Buyers: cross-chain treasury desks, OTC, bridge-integrated lenders, invoice factors.

Midnight's own 2026 priorities name "institutional execution" and "programmable compliance".

We attacked our own contract

An adversarial review found the merkle path was never bound to the recomputed leaf.

Before the fix: a proof claiming $2^{64}-1$ units against a real 1-unit attestation, to the wrong payee, past expiry, was ACCEPTED.

find_path is a WITNESS: it runs on the prover's machine and is not verified. Passing the leaf into it was a hint, not a constraint.

THE FIX, ONE LINE

```
assert(disclose(path.leaf == leaf), "merkle path does not open the claimed leaf");
```

THE EXPLOIT IS NOW A PERMANENT REGRESSION TEST

```
attestor is the deployer    : true
stranger can write registry: false
deployer can write registry: true
```

Disclosed rather than quietly patched. A security claim that has never been attacked is not evidence.

Aval

Act on a fact before that fact is public.

github.com/Leihyn/aval

Apache-2.0 · 2 circuits · 4 proving keys · 25 passing tests

Compact 0.26.0 · compiler 0.34.0 · runtime 0.19.0

No Docker, no proof server, no wallet required to verify